# PLAN-07 - Migration and Backward Compatibility

POS-DGFY - Global Location Switching | Plan v1.0 | Revision 2026-09-27

| Field | Value |
|---|---|
| Project | POS-DGFY - Global Location Switching |
| Plan version | 1.0 |
| Section | PLAN-07 - Migration and Backward Compatibility |
| Revision date | 2026-09-27 |
| Plan status | NOT EXECUTION READY |
| Purpose | Define discovery-first schema work, compatibility stages, historical protection, and migration exit criteria. |
| Dependencies | PLAN-03 through PLAN-06; ADR 0009 additive rollout policy; Phase 355 backend plan. |

## Migration position

**No migration is approved yet: First audit existing tables, indexes, constraints, repositories, and queries for vouchers, price lists, affiliates, settings, reports, and audit. Reuse existing location fields and transition records wherever present.**

### Required discovery

- Map every affected entity to tenant, location, terminal, shift, and historical location fields.
- Find all APIs accepting location_id or equivalent filters and classify whether they derive or trust it.
- Find all reports and exports that aggregate without an explicit location scope.
- Confirm whether settings are stored by tenant, branch, terminal, or mixed scope.
- Confirm voucher, price-list, and affiliate ownership and eligibility models.

### Safe migration sequence

| Stage | Action | Exit evidence |
|---|---|---|
| Prepare | Add compatible indexes/columns only when justified; keep old reads/writes working. | Migration tests and schema review pass. |
| Backfill | Populate location metadata from authoritative terminal, shift, transaction, or grant sources. | Row counts and ownership parity pass. |
| Validate | Compare old and new aggregates, reports, and ownership. | No unexplained drift. |
| Switch reads | Enable location-aware reads behind a feature flag. | Pilot reads match expected data. |
| Switch writes | Enable location-aware writes and audit. | Integration and reconciliation tests pass. |
| Observe | Monitor errors, totals, stock, permissions, and audit events. | Observation window passes. |
| Clean up | Remove compatibility code only after explicit approval. | Rollback window closed. |

### Backward compatibility

- Existing terminals and current shift flows must continue working when the feature flag is off.
- Older clients must fail safely or remain on compatible APIs; do not require a new client for an existing safe path without a migration plan.
- Historical reports must remain stable and must not be recomputed from later branch configuration.
- Generated build artifacts and unrelated dirty-worktree changes are not migration inputs.

## Migration exit criteria

- Representative tenant data has been migrated in rehearsal.
- Foreign keys, unique constraints, indexes, ownership, null handling, reports, and performance are verified.
- Rollback and reconciliation are tested without deleting historical data.